

SUY DIỄN QUY MÔ CÔNG NGHIỆP (INDUSTRIAL-STRENGTH INFERENCE)

CHƯƠNG 9.5–6, CHƯƠNG 8.1 VÀ 10.2–3

Nội dung

- ◇ Tính đầy đủ (Completeness)
- ◇ Phân giải (Độ phân giải)
- ◇ Lập trình logic (Lập trình logic)

Tính đầy đủ trong FOL

Thủ tục i là đầy đủ khi và chỉ khi

$$KB \vdash_i \alpha \quad \text{mỗi khi} \quad KB \models \alpha$$

Suy diễn tiến và lùi là đầy đủ cho KB Horn

Nhưng không đủ logic cho một bậc tổng hợp

Ví dụ, từ

$$Ti\ddot{y}ns(x) \Rightarrow Tri\grave{y}nh\ddot{O}echuy\grave{u}nm_ncao(x)$$

$$_ngph\check{c}iTi\ddot{y}ns(x) \Rightarrow Thunh\grave{h}ps\grave{a}m(x)$$

$$Cao(x) \Rightarrow Gi\grave{a}u(x)$$

$$Thunh\grave{h}ps\grave{a}m(x) \Rightarrow Gi\grave{a}u(x)$$

lẽ ra phải có thể suy ra $Rich(Me)$, nhưng FC/BC sẽ không làm được điều đó

Có tồn tại một thuật toán đầy đủ không?

Lịch sử sum họp của việc suy luận

450TCN	Stoics	logic mệnh đề, suy diễn (có thể)
322TCN	Aristotle	“tam đoạn luận” (quy tắc suy diễn), lượng từ
1565	Cardano	lý thuyết xác suất (logic mệnh đề + độ bất định)
1847	Boole	logic mệnh đề (một lần nữa)
1879	Frege	logic bậc một
1922	Wittgenstein	chứng minh bằng bảng chân lý
1930	Gödel	$\exists$ thuật toán đầy đủ cho FOL
1930	Herbrand	thuật toán đầy đủ cho FOL (rút gọn về mệnh đề)
1931	Gödel	$\neg\exists$ thuật toán đầy đủ cho số học
1960	Davis/Putnam	thuật toán “thực tiễn” cho mệnh đề logic
1965	Robinson	thuật toán “thực tiễn” cho FOL—giải phân tích

Phân tích (Độ phân giải)

Kéo theo logic bậc một chỉ là nửa quyết định (semidecidable):

có thể tìm thấy bằng chứng của α if $KB \models \alpha$

không thể luôn chứng minh rằng $KB \not\models \alpha$

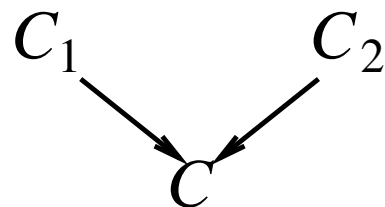
So sánh với vấn đề dừng (Vấn đề tạm dừng): thủ tục chứng minh có thể sắp xếp dừng với sự cố hoặc thất bại hoặc có thể diễn ra mãi mãi

Phân tích là một thủ tục bác bỏ (bác bỏ):

để chứng minh $KB \models \alpha$, chỉ ra rằng $KB \wedge \neg\alpha$ là không thỏa mãn được

Phân giải sử dụng $KB, \neg\alpha$ dưới dạng chuẩn hội (CNF) (phép hội của các mệnh đề)

Quy tắc suy diễn phân giải hợp hai mệnh đề để tạo ra một mệnh đề mới:



Sự suy diễn tiếp continue cho đến khi một mệnh đề trống (mệnh đề trống) được suy ra (mâu thuẫn)

Quy tắc suy diễn phân giải

Phiên bản mệnh đề cơ bản:

$$\frac{\alpha \vee \beta, \quad \neg\beta \vee \gamma}{\alpha \vee \gamma} \quad \text{hoặc tương đương} \quad \frac{\neg\alpha \Rightarrow \beta, \quad \beta \Rightarrow \gamma}{\neg\alpha \Rightarrow \gamma}$$

Phiên bản đầy đủ cấp độ:

$$\frac{\begin{array}{c} p_1 \vee \dots \vee p_j \vee \dots \vee p_m, \\ q_1 \vee \dots \vee q_k \vee \dots \vee q_n \end{array}}{(p_1 \vee \dots \vee p_{j-1} \vee p_{j+1} \vee \dots \vee p_m \vee q_1 \vee \dots \vee q_{k-1} \vee q_{k+1} \vee \dots \vee q_n)\sigma}$$

trong đó $p_j\sigma = \neg q_k\sigma$

Ví dụ:

$$\frac{\begin{array}{c} \neg Rich(x) \vee Unhappy(x) \\ Gi\ddot{a}u(T_i) \end{array}}{Kh_ngvui(T_i)}$$

với $\sigma = \{x/Me\}$

Dạng hội chuẩn (Conjunctive Normal Form)

Literal = câu nguyên thủy (có thể được phủ định), ví dụ: $\neg Rich(Me)$

Mệnh đề (Clause) = phép tuyển các chữ, ví dụ:

$\neg Rich(Me) \vee Unhappy(Me)$

KB được phép của các mệnh đề

Bất kỳ KB FOL nào cũng có thể được chuyển đổi thành CNF như sau:

1. Thay thế $P \Rightarrow Q$ bằng $\neg P \vee Q$
2. Chuyển $\neg$ vào trong, ví dụ: $\neg \forall x P$ trở thành $\exists x \neg P$
3. Chuẩn hóa các biến đổi, ví dụ: $\forall x P \vee \exists x Q$ trở thành $\forall x P \vee \exists y Q$
4. Chuyển lượng từ sang trái theo thứ tự, ví dụ: $\forall x P \vee \exists x Q$ trở thành $\forall x \exists y P \vee Q$
5. Loại bỏ $\exists$ bằng phương pháp Skolemization (slide tiếp theo)
6. Bỏ các từ phổ dụng
7. Phân phối $\wedge$ qua $\vee$, ví dụ: $(P \wedge Q) \vee R$ trở thành $(P \vee R) \wedge (Q \vee R)$

Phương pháp Skolemization

$\exists x \text{ Rich}(x)$ trở thành $\text{Rich}(G1)$ trong đó $G1$ là một “hằng số Skolem” mới

$$\exists k \frac{d}{dy}(k^y) = k^y \text{ trở thành } \frac{d}{dy}(e^y) = e^y$$

Khó hơn khi $\exists$ nằm bên trong $\forall$

Ví dụ: “Mọi người đều có một trái tim”

$$\forall x \text{ Person}(x) \Rightarrow \exists y \text{ Heart}(y) \wedge \text{Has}(x, y)$$

Sai:

$$\forall x \text{ Person}(x) \Rightarrow \text{Heart}(H1) \wedge \text{Has}(x, H1)$$

Đúng:

$$\forall x \text{ Person}(x) \Rightarrow \text{Heart}(H(x)) \wedge \text{Has}(x, H(x))$$

trong đó H là một ký hiệu mới (“hàm Skolem”)

Các đối số của hàm Skolem: Tất cả các biến số từ phổ dụng bao quanh

Chứng minh phân giải

Để chứng minh α :

- định nghĩa nó
- chuyển đổi sang CNF
- thêm vào CNF KB
- suy diễn ra ổn định

Ví dụ: để chứng minh $Rich(me)$, thêm $\neg Rich(me)$ vào CNF KB

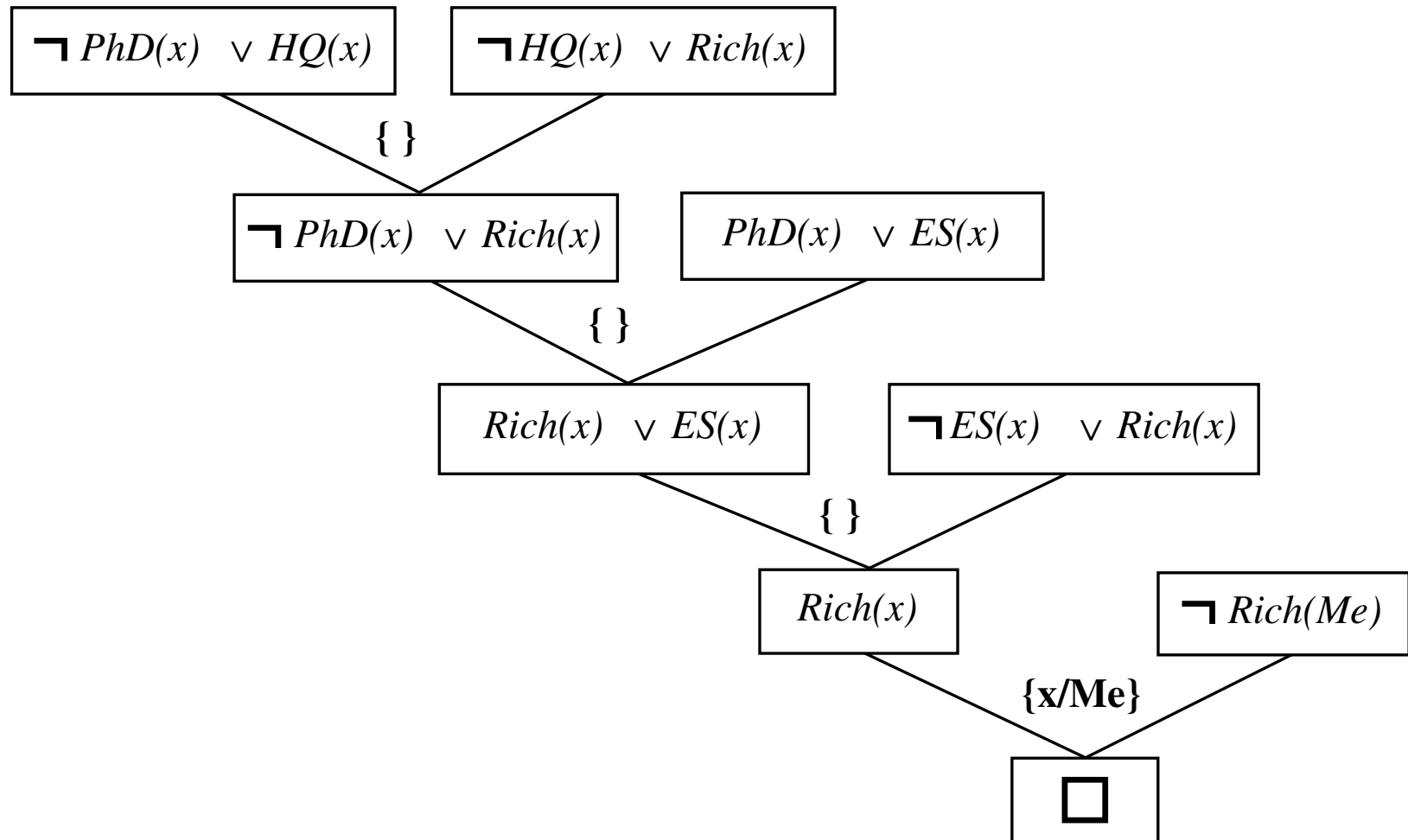
$\neg ngph\check{c}iTi\ddot{y}ns"(x) \vee C\hat{a}tri\eta nh\ddot{O}e\hat{c}ao(x)$

$Ti\ddot{y}ns"(x) \vee Th\eta nh\grave{l}ps\hat{m}(x)$

$\neg ng\ddot{O}i\acute{u}uch\grave{u}n\hat{c}ao(x) \vee Rich(x)$

$\neg ngph\check{c}iTh\eta nh\grave{l}ps\hat{m}(x) \vee Rich(x)$

Chứng minh phân giải



Logic trình nhập

Khẩu hiệu: tính toán là suy diễn trên logic KB

Lập trình logic

1. Xác định vấn đề
2. Thu thập thông tin
3. Nghĩ giải pháp
4. Mã hóa thông tin vào KB
5. Mã hóa ví dụ vấn đề thành sự kiện
6. Hỏi các truy vấn
7. Tìm các lỗi sự kiện

Lập trình thông thường

- Xác định vấn đề
Thu thập thông tin
Tìm ra giải pháp
Giải pháp lập trình
Mã hóa ví dụ vấn đề thành dữ liệu
Áp dụng chương trình vào dữ liệu
Loại bỏ các lỗi thủ tục

Tránh dễ dàng gỡ lỗi *Capital(NewYork, US)* hơn $x := x + 2$!

Hệ thống Prolog

Cơ sở: suy diễn ngược với các mệnh đề Horn + các tính năng bổ sung (chuông & còi)

Được sử dụng rộng rãi ở Châu Âu, Nhật Bản (cơ sở của dự án Thế hệ thứ 5)

Kỹ thuật biên dịch $\Rightarrow$ 10 triệu LIPS

Chương trình = tập các mệnh đề = `head :- Literal1, ... Litern.`

Hợp chất hiệu quả nhất bằng mã hóa mở (opencoding)

Truy xuất hiệu quả các mệnh đề phù hợp bằng liên kết trực tiếp

Suy diễn lùi từ trái sang phải, độ sâu ưu tiên

Các từ tích hợp cho số học v.v., ví dụ: `X is`

`Y*Z+3Ni1012]` Giả định thế giới đóng ("phủ định như thất bại")

ví dụ: `not PhD(X)` thành công nếu `PhD(X)` thất bại

Ví dụ Prolog

Tìm kiếm độ sâu ưu tiên từ trạng thái bắt đầu X:

```
dfs(X) :- goal(X).
```

```
dfs(X) :- successor(X,S),dfs(S).
```

Không cần lặp qua S: next thành công với mỗi trường hợp

Nối hai danh sách để tạo ra danh sách thứ ba:

```
append([],Y,Y).
```

```
append([X|L],Y,[X|Z]) :- append(L,Y,Z).
```

```
query:    append(A,B,[1,2]) ?
```

```
answers:  A=[]      B=[1,2]
```

```
          A=[1]     B=[2]
```

```
          A=[1,2]   B=[]
```